

Competitive Programming

Section 45 Editorial: Flows and Matchings

Competitive Programming Course

2026-07-10

Table of contents

1	Editorial overview	3
1.1	Problem-specific proof map	3
1.2	Detailed solution guide	3
2	A. Max Flow	5
2.1	Restatement	5
2.2	Observations and algorithm	5
2.3	Correctness proof	5
2.4	Complexity	5
2.5	Full C++ solution	5
2.6	Full Python solution	5
2.7	Tests that matter	6
3	B. Bipartite Matching	7
3.1	Restatement	7
3.2	Observations and algorithm	7
3.3	Correctness proof	7
3.4	Complexity	7
3.5	Full C++ solution	7
3.6	Full Python solution	7
3.7	Tests that matter	8
4	C. Min Cut Reachable Side	9
4.1	Restatement	9
4.2	Observations and algorithm	9
4.3	Correctness proof	9
4.4	Complexity	9
4.5	Full C++ solution	9
4.6	Full Python solution	9
4.7	Tests that matter	10
5	What to carry forward	11
6	Testing discussion	12

1 Editorial overview

1.1 Problem-specific proof map

A. Max Flow. Dinic augments only along residual paths, preserving capacity and flow conservation. When no residual s -to- t path remains, reachable residual vertices define a cut saturated by the flow, proving optimality.

B. Bipartite Matching. Unit-capacity source-left, compatibility, and right-sink edges make integral flow paths correspond one-for-one with matched pairs. Thus the maximum flow value is the maximum matching size.

C. Min Cut Reachable Side. After maximum flow, a residual BFS reaches exactly the source side of the minimum cut: any forward edge leaving it has zero residual capacity, while all reachable edges stay on that side.

1.2 Detailed solution guide

A. Maximum Flow. Dinic's algorithm works on the residual graph. A BFS builds levels using only edges with positive residual capacity. A DFS then sends flow only from level d to level $d+1$, which forms a blocking flow for the current level graph. When the sink is unreachable in BFS, there is no augmenting path left. By the max-flow min-cut theorem, the current flow is maximum. The reverse edge stores the option to cancel earlier choices, which is essential for correctness.

B. Bipartite Matching. Model each left vertex as choosing at most one right vertex. The DFS tries to match a left vertex directly, and if the right vertex is already used, it recursively asks the previous owner to move elsewhere. A successful recursive move creates an augmenting path and increases the matching size by one. When no DFS from any unmatched left vertex succeeds, no augmenting path remains, so the matching is maximum.

C. Minimum Cut After Flow. After computing max flow, run a reachability search from the source in the final residual graph. Let S be the reachable set. No residual edge of positive capacity crosses from S to $V \setminus S$; otherwise the endpoint would be reachable. Thus all original capacity across that cut is saturated, and its value equals the max flow. The printed cut is therefore a minimum cut.

Common pitfalls. Always update reverse edges together with forward edges. For matching, reset the visited array for each DFS root. For min cut, use the residual graph after max flow, not the original graph.

The three exercises are different views of augmenting paths.

- A implements Dinic. BFS builds a residual level graph; DFS sends blocking flow along increasing levels; reverse edges allow rerouting earlier choices.
- B solves bipartite matching with augmenting DFS. A right vertex can be reassigned if its current left partner can find another route.

- C runs max flow, then explores the residual graph from the source. The reachable set is the source side of a minimum cut.

For max-flow correctness, the key terminal condition is: if the sink is not reachable in the residual graph, there is no augmenting path, so the current flow is maximum.

2 A. Max Flow

2.1 Restatement

Given a directed capacitated graph from node 1 to node n, print the maximum flow.

2.2 Observations and algorithm

Dinic works on residual edges. BFS assigns shortest residual levels; DFS pushes only along positive edges from level d to d+1 until that level graph blocks all source-sink paths. Update the reverse residual edge whenever flow is pushed.

2.3 Correctness proof

Residual capacities preserve capacity and conservation. When BFS cannot reach sink, no augmenting path remains; max-flow/min-cut proves the flow maximum. QED.

2.4 Complexity

Dinic is $O(V^2 E)$ in the general integral-capacity bound, with $O(V+E)$ graph storage.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; struct Dinic{struct E{int v,rev; long long c;};int n;vector<vector<E>>>g;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m;if(!(cin>>n>>m)) return 0;Dinic g(n);
```

2.6 Full Python solution

```
import sys
from collections import deque
def dinic(n,edges,s,t,cut=False):
    g=[[] for _ in range(n)]
    def add(u,v,c):
        g[u].append([v,c,len(g[v])]); g[v].append([u,0,len(g[u])-1])
    for u,v,c in edges: add(u,v,c)
```

```

flow=0
while True:
    level=[-1]*n; level[s]=0; dq=deque([s])
    while dq:
        u=dq.popleft()
        for v,c,rev in g[u]:
            if c and level[v]<0: level[v]=level[u]+1; dq.append(v)
    if level[t]<0: break
    it=[0]*n
    def dfs(u,f):
        if u==t: return f
        for i in range(it[u],len(g[u])):
            it[u]=i; v,c,rev=g[u][i]
            if c and level[v]==level[u]+1:
                ret=dfs(v,min(f,c))
                if ret: g[u][i][1]-=ret; g[v][rev][1]+=ret; return ret
        return 0
    while True:
        pushed=dfs(s,10**18)
        if not pushed: break
        flow+=pushed
    if cut:
        seen=[False]*n; dq=deque([s]); seen[s]=True
        while dq:
            u=dq.popleft()
            for v,c,rev in g[u]:
                if c and not seen[v]: seen[v]=True; dq.append(v)
        return flow,seen
    return flow,None
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,m=data[0],data[1]; idx=2; edges=[]
    for _ in range(m): edges.append((data[idx]-1,data[idx+1]-1,data[idx+2])); idx+=3
    print(dinic(n,edges,0,n-1)[0])
if __name__=="__main__": main()

```

2.7 Tests that matter

Test no path, parallel edges, a bottleneck, and a rerouting-required network.

3 B. Bipartite Matching

3.1 Restatement

Given edges between left and right parts, print the maximum matching size.

3.2 Observations and algorithm

Use augmenting DFS. A right vertex is free, or its current left owner must first move through an alternate edge. The successful recursion chain is one augmenting path. For L1-R1,R2 and L2-R1, reroute L1 to R2 then match L2-R1.

3.3 Correctness proof

Each success increases matching size by one. If every unmatched left root fails, there is no augmenting path, so the matching is maximum. QED.

3.4 Complexity

The Kuhn DFS implementation is $O(L \cdot E)$ worst case and uses $O(L+R+E)$ memory.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m,e;if(!
```

3.6 Full Python solution

```
import sys, collections
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,m,e=data[0],data[1],data[2]; adj=[[] for _ in range(n)]; idx=3
    for _ in range(e): adj[data[idx]-1].append(data[idx+1]-1); idx+=2
    match=[-1]*m
    def dfs(u,vis):
```

```

    for v in adj[u]:
        if vis[v]: continue
        vis[v]=1
        if match[v]==-1 or dfs(match[v],vis): match[v]=u; return True
    return False
ans=0
for u in range(n): ans+=dfs(u,[0]*m)
print(ans)
if __name__=="__main__": main()

```

3.7 Tests that matter

Reset visited-right per root. Test no edges, a reroute case, duplicates, and unequal part sizes.

4 C. Min Cut Reachable Side

4.1 Restatement

After max flow from 1 to n, print the flow value and the source-side reachable vertices in the residual graph.

4.2 Observations and algorithm

After max flow, BFS from source through positive final residual edges. Print the reachable vertices as S together with the flow value.

4.3 Correctness proof

No residual forward edge leaves S, so every original edge from S is saturated. That cut has capacity equal to the maximum flow and is a minimum cut. QED.

4.4 Complexity

The extra residual BFS is $O(V+E)$ after the max-flow computation.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; struct Dinic{struct E{int v,rev; long long c;};int n;vector<vector<E>>>g;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m;if(!(cin>>n>>m)) return 0;Dinic g(n);
```

4.6 Full Python solution

```
import sys
from collections import deque
def dinic(n,edges,s,t,cut=False):
    g=[[] for _ in range(n)]
    def add(u,v,c):
        g[u].append([v,c,len(g[v])]); g[v].append([u,0,len(g[u])-1])
    for u,v,c in edges: add(u,v,c)
```

```

flow=0
while True:
    level=[-1]*n; level[s]=0; dq=deque([s])
    while dq:
        u=dq.popleft()
        for v,c,rev in g[u]:
            if c and level[v]<0: level[v]=level[u]+1; dq.append(v)
    if level[t]<0: break
    it=[0]*n
    def dfs(u,f):
        if u==t: return f
        for i in range(it[u],len(g[u])):
            it[u]=i; v,c,rev=g[u][i]
            if c and level[v]==level[u]+1:
                ret=dfs(v,min(f,c))
                if ret: g[u][i][1]-=ret; g[v][rev][1]+=ret; return ret
        return 0
    while True:
        pushed=dfs(s,10**18)
        if not pushed: break
        flow+=pushed
    if cut:
        seen=[False]*n; dq=deque([s]); seen[s]=True
        while dq:
            u=dq.popleft()
            for v,c,rev in g[u]:
                if c and not seen[v]: seen[v]=True; dq.append(v)
        return flow,seen
    return flow,None
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,m=data[0],data[1]; idx=2; edges=[]
    for _ in range(m): edges.append((data[idx]-1,data[idx+1]-1,data[idx+2])); idx+=3
    flow,seen=dinic(n,edges,0,n-1,True); print(flow); print(' '.join(str(i+1) for i,x in enumerate(seen) if x))
if __name__=="__main__": main()

```

4.7 Tests that matter

Test source-only reachability, zero flow, and parallel cut edges. Use the final residual graph, never the original graph.

5 What to carry forward

Advanced algorithms become manageable when you state the invariant first, then code only the operations needed to preserve it.

6 Testing discussion

For tiny random networks, compare with a slow augmenting-path implementation; for matching, compare with brute force over small left vertices. Verify cut output by checking every residual-reachable edge and summing original capacities leaving the reached set. Run:

```
CP_TARGET=solution python3 sections/45_flows_and_matchings/check.py
```